

An Implementable Fixed-Branch Squared Tensor-Train Filter

BayesFilter P15 Zhao–Cui TT Specification

2026-05-31

Abstract

This note specifies one concrete fixed-branch squared tensor-train filtering algorithm and its analytical same-scalar gradient. It is inspired by the sequential squared tensor-train construction of Zhao and Cui, but it is not a description of every adaptive choice in their research code. The purpose is more modest and more operational: define a single value path that a reader can implement without opening any other source. The fixed path uses affine domain maps, normalized Legendre bases, deterministic Halton fitting points, fixed ranks, ridge alternating least squares, a uniform defensive density, a frozen shift, explicit squared-TT marginalization, and a finite-difference parity test. The resulting scalar is an approximate likelihood; the gradient derived here differentiates that approximate scalar on the saved branch.

Contents

1	What This Specification Implements	2
2	State-Space Filtering Object And Approximate Likelihood	3
3	Fixed Computational Domain And Reference Measure	3
4	Legendre Basis, Evaluation Recurrence, And Mass Matrices	4
5	Tensor-Train Functions And Squared-TT Densities	5
6	Fixed Fitting Design Points	5
7	Fixed-Rank Ridge ALS Construction Of The Square-Root TT	5
8	Normalizer, Defensive Density, Shift, And Approximate Log-Likelihood	6
9	Marginalization In Squared-TT Form And The Next Filter Object	7
10	Saved Branch Data Structures	9
11	Forward Filtering Algorithm In Pseudocode	9
12	Derivative Of Target Values And Previous-Filter Evaluations	10
13	Derivative Through Fixed ALS Sweeps	10
14	Derivative Of Mass Contractions, Marginals, And The Likelihood Scalar	11

15 Complete Gradient Algorithm In Pseudocode	12
16 Numerical Stabilization And Failure Diagnostics	12
17 Finite-Difference Parity Test	13
18 Minimal Runnable Example	13
19 Relation To Zhao–Cui And What Remains To Be Tested	14
A Source And Code Anchors	14
B Symbol Table And Array Shapes	15

1 What This Specification Implements

The algorithm in this note takes a state-space model and produces an approximate filtering likelihood. The model has state x_t , optional static parameter ϑ , observation y_t , transition density $f_\alpha(x_t \mid x_{t-1}, \vartheta)$, and observation density $g_\alpha(y_t \mid x_t, \vartheta)$. The external parameter α is the quantity with respect to which the approximate likelihood will be differentiated.

At each time t , the method constructs an approximation to the unnormalized filtering density over the augmented variable

$$r_t = (x_t, \vartheta, x_{t-1}) \in \mathbb{R}^D. \quad (1)$$

The approximation has the form

$$\hat{q}_t(z; \alpha; B_t) = \phi_t(z; \alpha; B_t)^2 + \tau_t 2^{-D}, \quad z \in [-1, 1]^D, \quad (2)$$

where B_t denotes the fixed branch data. The square-root tensor train ϕ_t is fitted by a fully specified fixed-rank ridge ALS procedure. The defensive term $\tau_t 2^{-D}$ is a small uniform density floor on the reference box.

The one-step approximate evidence contribution is

$$\hat{z}_t(\alpha; B_t) = \int_{[-1, 1]^D} \hat{q}_t(z; \alpha; B_t) \, dz = R_{t,0}(\alpha; B_t) + \tau_t, \quad (3)$$

and the declared approximate log-likelihood is

$$\hat{\ell}_T^{\text{TT}}(\alpha; B) = \sum_{t=1}^T \{\log(R_{t,0}(\alpha; B_t) + \tau_t) - c_t\}. \quad (4)$$

The constants c_t are frozen stabilizing shifts stored in the branch. The gradient derived below is the derivative of (4) for the saved branch $B = (B_1, \dots, B_T)$. It is not the derivative of an adaptive algorithm whose ranks, fitting points, shifts, domains, or truncations change with α .

This distinction matters. A filtering algorithm may adapt its numerical representation as the parameter changes. That can be useful for accuracy, but then the formula being evaluated changes. A classical analytical gradient must differentiate one declared formula. P15 therefore freezes the branch objects and differentiates the resulting approximate scalar.

2 State-Space Filtering Object And Approximate Likelihood

Assume that before observing y_t , the previous approximate filter is a normalized density $\hat{p}_{t-1}(x_{t-1}, \vartheta; \alpha)$. The exact Bayesian one-step unnormalized joint density would be

$$q_t(r_t; \alpha) = \hat{p}_{t-1}(x_{t-1}, \vartheta; \alpha) f_\alpha(x_t | x_{t-1}, \vartheta) g_\alpha(y_t | x_t, \vartheta). \quad (5)$$

Its integral is

$$Z_t(\alpha) = \int q_t(x_t, \vartheta, x_{t-1}; \alpha) dx_t d\vartheta dx_{t-1}. \quad (6)$$

Integrating first over x_{t-1} gives the prediction density, and then integrating against the likelihood gives the innovation evidence:

$$Z_t(\alpha) = \int g_\alpha(y_t | x_t, \vartheta) \left[\int f_\alpha(x_t | x_{t-1}, \vartheta) \hat{p}_{t-1}(x_{t-1}, \vartheta; \alpha) dx_{t-1} \right] dx_t d\vartheta. \quad (7)$$

The updated filter is the marginal

$$p_t(x_t, \vartheta; \alpha) = \int \frac{q_t(x_t, \vartheta, x_{t-1}; \alpha)}{Z_t(\alpha)} dx_{t-1}. \quad (8)$$

The P15 method replaces q_t by a nonnegative finite-dimensional approximation $\hat{q}_t$, integrates $\hat{q}_t$ to get $\hat{z}_t$, and marginalizes $\hat{q}_t/\hat{z}_t$ to get the next approximate filter. Thus the connection to filtering is not decorative: the tensor train is only a way to compute the two integrals in (7) and (8).

3 Fixed Computational Domain And Reference Measure

Choose a hyperrectangle

$$\Omega_t = \prod_{k=1}^D [\ell_{t,k}, u_{t,k}] \quad (9)$$

with finite endpoints. The reference coordinate is $z \in [-1, 1]^D$. Define

$$a_{t,k} = \frac{u_{t,k} + \ell_{t,k}}{2}, \quad h_{t,k} = \frac{u_{t,k} - \ell_{t,k}}{2}, \quad r_{t,k} = a_{t,k} + h_{t,k} z_k. \quad (10)$$

Let $\Psi_t(z)$ denote this affine map. The Jacobian determinant is

$$J_t = \prod_{k=1}^D h_{t,k}. \quad (11)$$

The transformed unnormalized density is

$$\tilde{q}_t(z; \alpha) = q_t(\Psi_t(z); \alpha) J_t. \quad (12)$$

Proposition 1 (Affine-domain transformation). *For any integrable q_t on Ω_t ,*

$$\int_{\Omega_t} q_t(r; \alpha) dr = \int_{[-1, 1]^D} \tilde{q}_t(z; \alpha) dz. \quad (13)$$

Proof. Equation (10) is diagonal affine. Hence $dr = J_t dz$. Substituting $r = \Psi_t(z)$ gives

$$\int_{\Omega_t} q_t(r; \alpha) dr = \int_{[-1,1]^D} q_t(\Psi_t(z); \alpha) J_t dz, \quad (14)$$

which is (13). $\square$

For the fixed-branch derivative, $\ell_{t,k}$, $u_{t,k}$, $a_{t,k}$, $h_{t,k}$, and J_t are frozen. If a future implementation adapts the box with α , that is a different scalar and requires additional boundary and Jacobian derivatives.

4 Legendre Basis, Evaluation Recurrence, And Mass Matrices

On each coordinate, use normalized Legendre functions

$$b_{k,a}(z) = \sqrt{\frac{2a+1}{2}} P_a(z), \quad a = 0, \dots, p_k - 1, \quad (15)$$

where P_a is the degree- a Legendre polynomial. The implementation evaluates them by

$$P_0(z) = 1, \quad P_1(z) = z, \quad (16)$$

$$(a+1)P_{a+1}(z) = (2a+1)zP_a(z) - aP_{a-1}(z), \quad a \geq 1. \quad (17)$$

For a vector $z_k^{(1)}, \dots, z_k^{(N)}$, store the basis matrix

$$B_k[j, a] = b_{k,a}(z_k^{(j)}). \quad (18)$$

Proposition 2 (Legendre mass matrix). *The one-dimensional mass matrix*

$$M_k[a, b] = \int_{-1}^1 b_{k,a}(z) b_{k,b}(z) dz \quad (19)$$

equals the identity matrix.

Proof. Legendre orthogonality gives

$$\int_{-1}^1 P_a(z) P_b(z) dz = \frac{2}{2a+1} \mathbf{1}_{\{a=b\}}. \quad (20)$$

Multiplying by the normalization constants in (15) gives

$$M_k[a, b] = \sqrt{\frac{2a+1}{2}} \sqrt{\frac{2b+1}{2}} \frac{2}{2a+1} \mathbf{1}_{\{a=b\}} = \mathbf{1}_{\{a=b\}}. \quad (21)$$

$\square$

This identity is why the P15 reference path uses normalized Legendre bases. Mass contractions then become ordinary Euclidean coefficient contractions.

5 Tensor-Train Functions And Squared-TT Densities

A scalar function on the reference box is represented by cores

$$C_{k,a} \in \mathbb{R}^{r_{k-1} \times r_k}, \quad r_0 = r_D = 1, \quad (22)$$

and one-dimensional matrix-valued factors

$$G_k(z_k) = \sum_{a=0}^{p_k-1} C_{k,a} b_{k,a}(z_k). \quad (23)$$

The tensor-train function is

$$\phi(z) = G_1(z_1)G_2(z_2) \cdots G_D(z_D), \quad (24)$$

where the product is a 1×1 scalar.

The density approximation uses the square

$$\hat{q}_t(z) = \phi_t(z)^2 + \tau_t 2^{-D}. \quad (25)$$

The square guarantees nonnegativity. The uniform term prevents a zero density floor on the compact reference box. It does not make a bad fit accurate; it only ensures that the approximate density remains a proper nonnegative object after normalization.

6 Fixed Fitting Design Points

The fitting design is deterministic. Let $p_1, \dots, p_D$ be the first D primes. If

$$j = \sum_{m=0}^{M(j,p)} d_m p^m, \quad d_m \in \{0, \dots, p-1\}, \quad (26)$$

define the radical inverse

$$\text{radinv}_p(j) = \sum_{m=0}^{M(j,p)} d_m p^{-(m+1)}. \quad (27)$$

For $j = 1, \dots, N_{\text{fit}}$, the fitting point is

$$z_k^{(j)} = 2 \text{radinv}_{p_k}(j) - 1, \quad k = 1, \dots, D. \quad (28)$$

The least-squares weight matrix is

$$W = \frac{1}{N_{\text{fit}}} I. \quad (29)$$

The points, primes, and weights are saved branch objects. They are not resampled when evaluating $\hat{\ell}_T^{\text{TT}}(\alpha \pm h e_i; B)$ in a finite-difference check.

7 Fixed-Rank Ridge ALS Construction Of The Square-Root TT

The target values at the fixed design points are

$$y_j(\alpha) = \exp(c_t/2) \sqrt{\tilde{q}_t(z^{(j)}; \alpha)}, \quad j = 1, \dots, N_{\text{fit}}. \quad (30)$$

The stabilizing shift c_t is frozen and defined at a nominal value α_0 by

$$c_t = - \max_{1 \leq j \leq N_{\text{fit}}} \log \tilde{q}_t(z^{(j)}; \alpha_0). \quad (31)$$

The target values therefore remain numerically moderate near the branch point. The shift enters the declared scalar as $-c_t$, as in (4).

Fix ranks $r_0 = r_D = 1$ and $r_1, \dots, r_{D-1}$. Fix ridge $\rho > 0$, number of bidirectional ALS sweeps S , and the following exact initialization. Let $e_1^{(m)}$ be the first coordinate vector in $\mathbb{R}^m$. The constant slice of core k is

$$C_{k,0}^{(0)} = \gamma_k e_1^{(r_{k-1})} (e_1^{(r_k)})^\top, \quad C_{k,a}^{(0)} = 0 \quad (a \geq 1), \quad (32)$$

where

$$\gamma_1 = \bar{y}, \quad \gamma_k = 1 \quad (k = 2, \dots, D), \quad \bar{y} = \frac{1}{N_{\text{fit}}} \sum_{j=1}^{N_{\text{fit}}} y_j. \quad (33)$$

Thus only the first rank channel is active initially. All other rank channels start at zero and can become nonzero only through the declared ridge ALS updates. This convention fixes the branch even when ranks exceed one.

During an ALS update of core k , all other cores are fixed. Define the left environment

$$L_j^{(k)} = G_1(z_1^{(j)}) \cdots G_{k-1}(z_{k-1}^{(j)}) \in \mathbb{R}^{1 \times r_{k-1}} \quad (34)$$

and the right environment

$$R_j^{(k)} = G_{k+1}(z_{k+1}^{(j)}) \cdots G_D(z_D^{(j)}) \in \mathbb{R}^{r_k \times 1}. \quad (35)$$

If $g_k = \text{vec}(C_{k,0}, \dots, C_{k,p_k-1})$ is ordered lexicographically in $(\ell_{\text{left}}, a, \ell_{\text{right}})$, with the right rank varying fastest, then the row that maps g_k to $\phi(z^{(j)})$ is

$$A_{j,\cdot}^{(k)} = (R_j^{(k)})^\top \otimes b_k(z_k^{(j)})^\top \otimes L_j^{(k)}. \quad (36)$$

The fixed ridge update is

$$N_k g_k = d_k, \quad N_k = (A^{(k)})^\top W A^{(k)} + \rho I, \quad d_k = (A^{(k)})^\top W y. \quad (37)$$

The algorithm applies updates for $k = 1, \dots, D$ and then $k = D, \dots, 1$, repeated for exactly S sweeps. There is no rank adaptation, no pivot selection, and no truncation in the declared branch.

8 Normalizer, Defensive Density, Shift, And Approximate Log-Likelihood

The defensive density is uniform:

$$\lambda(z) = 2^{-D}, \quad \int_{[-1,1]^D} \lambda(z) \, dz = 1. \quad (38)$$

Choose an $\epsilon_\tau > 0$ and set

$$\tau_t = \epsilon_\tau 2^D. \quad (39)$$

This makes the defensive floor equal to ϵ_τ :

$$\tau_t 2^{-D} = \epsilon_\tau. \quad (40)$$

For the primary P15 scalar, τ_t and c_t are fixed branch constants, so

$$\partial_i \tau_t = 0, \quad \partial_i c_t = 0. \quad (41)$$

To integrate ϕ_t^2 , start with $R_D = 1$ and compute backward

$$R_{k-1} = \sum_{a,b=0}^{p_k-1} M_k[a, b] C_{k,a} R_k C_{k,b}^\top. \quad (42)$$

Because $M_k = I$, this becomes

$$R_{k-1} = \sum_{a=0}^{p_k-1} C_{k,a} R_k C_{k,a}^\top. \quad (43)$$

Proposition 3 (Squared-TT mass contraction). *The recursion (42) satisfies*

$$R_0 = \int_{[-1,1]^D} \phi_t(z)^2 dz. \quad (44)$$

Proof. For the last coordinate,

$$\int G_D(z_D) G_D(z_D)^\top dz_D = \sum_{a,b} C_{D,a} C_{D,b}^\top \int b_{D,a}(z_D) b_{D,b}(z_D) dz_D, \quad (45)$$

which is R_{D-1} . Suppose R_k equals the suffix integral over coordinates $k+1, \dots, D$. Multiplying by $G_k(z_k)$ on the left and $G_k(z_k)^\top$ on the right and integrating z_k gives exactly (42). Induction to $k=1$ gives (44). $\square$

Thus

$$\hat{z}_t = R_0 + \tau_t, \quad \hat{\ell}_t^{\text{TT}} = \log(R_0 + \tau_t) - c_t. \quad (46)$$

9 Marginalization In Squared-TT Form And The Next Filter Object

Let K be the set of kept coordinates, such as x_t, ϑ , and O the set of old-state coordinates to integrate out. Define square cores

$$H_k(z_k) = G_k(z_k) \otimes G_k(z_k). \quad (47)$$

Their coefficient slices are

$$H_{k,(a,b)} = C_{k,a} \otimes C_{k,b}. \quad (48)$$

For an integrated coordinate $k \in O$, replace H_k by

$$\mathcal{I}_k = \int_{-1}^1 H_k(z_k) dz_k = \sum_{a,b} M_k[a, b] C_{k,a} \otimes C_{k,b}. \quad (49)$$

For a kept coordinate $k \in K$, evaluate $H_k(z_k)$. Contract the chain in the original order, using $\mathcal{I}_k$ for integrated coordinates. The square-term marginal numerator is

$$a_t^{\text{sq}}(z_K) = \int \phi_t(z)^2 dz_O. \quad (50)$$

The defensive contribution is

$$a_t^{\text{def}}(z_K) = \tau_t 2^{-D} \int_{[-1,1]^{|O|}} 1 \, dz_O = \tau_t 2^{-|K|}. \quad (51)$$

The stored numerator and filter are

$$a_t(z_K) = a_t^{\text{sq}}(z_K) + \tau_t 2^{-|K|}, \quad \hat{p}_t(z_K) = a_t(z_K) / \hat{z}_t. \quad (52)$$

Proposition 4 (Squared-TT marginalization). *Equation (52) is the marginal of $\hat{q}_t / \hat{z}_t$ over the old coordinates.*

Proof. For the square part, expand ϕ_t^2 as the product of square cores (47). Fubini's theorem allows each integrated coordinate $k \in O$ to be replaced by $\mathcal{I}_k$ in (49). Kept coordinates are not integrated and therefore remain as functions of z_K . This gives (50). The uniform defensive density is constant, so integrating $|O|$ coordinates multiplies it by $2^{|O|}$, giving (51). Dividing by $\hat{z}_t$ normalizes the marginal. $\square$

The next-step fitting routine needs pointwise values. The saved filter object therefore stores a concrete contraction scheme. First build the square-core coefficient slices

$$\mathcal{H}_{k,ab} = C_{k,a} \otimes C_{k,b} \in \mathbb{R}^{(r_{k-1}^2) \times (r_k^2)}. \quad (53)$$

For an integrated coordinate define the constant matrix

$$Q_k = \sum_{a,b} M_k[a,b] \mathcal{H}_{k,ab}. \quad (54)$$

For a kept coordinate and a point z_k , define

$$E_k(z_k) = \sum_{a,b} b_{k,a}(z_k) b_{k,b}(z_k) \mathcal{H}_{k,ab}. \quad (55)$$

Now set

$$T_k(z_K) = \begin{cases} E_k(z_k), & k \in K, \\ Q_k, & k \in O. \end{cases} \quad (56)$$

The square marginal is the scalar chain contraction

$$a_t^{\text{sq}}(z_K) = T_1(z_K) T_2(z_K) \cdots T_D(z_K). \quad (57)$$

The object also stores prefix and suffix contractions

$$P_0 = 1, \quad P_k = P_{k-1} T_k, \quad S_D = 1, \quad S_{k-1} = T_k S_k, \quad (58)$$

so that an implementation can evaluate the numerator in one left-to-right pass and, when needed, reuse environments for derivatives. The value and log-value evaluators are

$$a_t(z_K) = a_t^{\text{sq}}(z_K) + \tau_t 2^{-|K|}, \quad \log \hat{p}_t(z_K) = \log a_t(z_K) - \log \hat{z}_t. \quad (59)$$

For derivatives, store

$$\dot{\mathcal{H}}_{k,ab} = \dot{C}_{k,a} \otimes C_{k,b} + C_{k,a} \otimes \dot{C}_{k,b}, \quad (60)$$

then build $\dot{Q}_k$ and $\dot{E}_k(z_k)$ by replacing $\mathcal{H}$ with $\dot{\mathcal{H}}$ in (54) and (55). The derivative of the square marginal is

$$\dot{a}_t^{\text{sq}}(z_K) = \sum_{m=1}^D P_{m-1} \dot{T}_m S_m, \quad (61)$$

where $\dot{T}_m = \dot{E}_m$ for kept coordinates and $\dot{T}_m = \dot{Q}_m$ for integrated coordinates. The defensive kept term is fixed, so

$$\dot{a}_t(z_K) = \dot{a}_t^{\text{sq}}(z_K), \quad \partial_i \log \hat{p}_t(z_K) = \frac{\dot{a}_t(z_K)}{a_t(z_K)} - \frac{\dot{\hat{z}}_t}{\hat{z}_t}. \quad (62)$$

Equations (53)–(62) are the exact saved-filter representation used by the two-step reference example.

10 Saved Branch Data Structures

The implementation must save the following objects.

Object	Required fields
DomainMap	lower bounds ℓ_k , upper bounds u_k , centers a_k , half-widths h_k , $\log J$.
Basis1D	degree p_k , Legendre recurrence evaluator, mass matrix $M_k = I$.
FitDesign	Halton points, coordinate primes, uniform weights, N_{fit} .
TTCore	array with shape (r_{k-1}, p_k, r_k) .
TTFunction	ordered cores, basis objects, evaluation routine.
SquaredTTDensity	root TT, τ_t , uniform defensive density, c_t , R_0 , $\hat{z}_t$, square-core environments.
FilterObject	numerator representation a_t , normalizer $\hat{z}_t$, value, log-value, derivative, and log-derivative evaluators.
BranchObject	ordering, domain maps, basis degrees, ranks, design points, weights, ridge, sweep count, initialization, shift, τ_t , vectorization order, saved ALS systems.
SensitivityObject	derivatives of target values, cores, environments, normalizers, marginal numerator, and score terms.

Saving these objects is not optional. Without them, an implementation cannot prove that the derivative pass differentiates the same scalar as the forward pass.

11 Forward Filtering Algorithm In Pseudocode

F1. Choose and save B_t : domain map, Legendre degrees, ranks, Halton points, weights, ridge, sweep count, initialization rule, ϵ_τ , and c_t .

F2. At each fitting point $z^{(j)}$, compute

$$\tilde{q}_t(z^{(j)}; \alpha) = \hat{p}_{t-1}(\Psi_{t,\text{old}}(z^{(j)}); \alpha) f_\alpha(\Psi_{t,x}(z^{(j)}) \mid \Psi_{t,\text{old}}(z^{(j)})) g_\alpha(y_t \mid \Psi_{t,x}(z^{(j)})) J_t. \quad (63)$$

F3. Form $y_j = \exp(c_t/2) \sqrt{\tilde{q}_t(z^{(j)}; \alpha)}$.

F4. Run exactly S bidirectional ridge ALS sweeps using (36) and (37).

- F5.** Compute R_0 by (42) and $\hat{z}_t = R_0 + \tau_t$.
- F6.** Add $\log \hat{z}_t - c_t$ to the approximate log-likelihood.
- F7.** Build and save the next `FilterObject` by (52).

12 Derivative Of Target Values And Previous-Filter Evaluations

For a positive density value,

$$y_j = \exp(c_t/2) \sqrt{\tilde{q}_j}, \quad \dot{y}_j = \frac{1}{2} y_j \partial_i \log \tilde{q}_j, \quad (64)$$

because c_t is frozen. Here and below a dot denotes ∂_i . At time t ,

$$\partial_i \log \tilde{q}_t = \partial_i \log \hat{p}_{t-1} + \partial_i \log f_\alpha + \partial_i \log g_\alpha, \quad (65)$$

since the branch domain Jacobian is fixed.

The previous filter derivative is evaluated from its saved numerator:

$$\hat{p}_{t-1}(z_K) = a_{t-1}(z_K)/\hat{z}_{t-1}, \quad \partial_i \log \hat{p}_{t-1} = \frac{\dot{a}_{t-1}}{a_{t-1}} - \frac{\dot{\hat{z}}_{t-1}}{\hat{z}_{t-1}}. \quad (66)$$

This is the operational reason for saving derivative environments for the next filter object.

In code, the evaluation routine for a new kept-coordinate point performs:

- E1.** Evaluate the kept-coordinate basis vectors $b_k(z_k)$.
- E2.** Build $E_k(z_k)$ and $\dot{E}_k(z_k)$ from (55) and the derivative analogue.
- E3.** Use stored $Q_k, \dot{Q}_k$ for integrated coordinates.
- E4.** Form $T_k, \dot{T}_k$, prefix products P_k , and suffix products S_k by (58).
- E5.** Compute $a_t, \dot{a}_t, \log \hat{p}_t$, and $\partial_i \log \hat{p}_t$ by (59) and (62).

No hidden tensor-train operation remains in this evaluator; it is only matrix products in the squared ranks.

13 Derivative Through Fixed ALS Sweeps

Each stored ALS update solves

$$Ng = d, \quad N = A^\top W A + \rho I, \quad d = A^\top W y. \quad (67)$$

For the fixed branch, W, ρ , ranks, and fitting points are fixed. The matrices A may still depend on α because their rows contain environments from other cores, and those cores depend on previous ALS updates. Differentiating gives

$$\dot{N} = \dot{A}^\top W A + A^\top W \dot{A}, \quad \dot{d} = \dot{A}^\top W y + A^\top W \dot{y}. \quad (68)$$

Proposition 5 (Fixed-rank ridge ALS solve derivative). *If N is nonsingular, the derivative of the stored update is*

$$N\dot{g} = \dot{d} - \dot{N}g. \quad (69)$$

Proof. Differentiate $Ng = d$. The product rule gives

$$\dot{N}g + N\dot{g} = \dot{d}. \quad (70)$$

Rearranging gives (69). The ridge term helps make N nonsingular, but the implementation must still monitor conditioning. $\square$

The derivative of the design row follows from

$$A_{j,\cdot}^{(k)} = (R_j^{(k)})^\top \otimes b_k(z_k^{(j)})^\top \otimes L_j^{(k)}. \quad (71)$$

The basis values are fixed, so

$$\dot{A}_{j,\cdot}^{(k)} = (\dot{R}_j^{(k)})^\top \otimes b_k(z_k^{(j)})^\top \otimes L_j^{(k)} + (R_j^{(k)})^\top \otimes b_k(z_k^{(j)})^\top \otimes \dot{L}_j^{(k)}. \quad (72)$$

The environments use ordinary product rules. For example,

$$\dot{L}_j^{(k)} = \sum_{m < k} G_1 \cdots \dot{G}_m \cdots G_{k-1}. \quad (73)$$

14 Derivative Of Mass Contractions, Marginals, And The Likelihood Scalar

Differentiate the mass contraction (42). With fixed mass matrices,

$$\begin{aligned} \dot{R}_{k-1} = \sum_{a,b} M_k[a,b] & \left[\dot{C}_{k,a} R_k C_{k,b}^\top + C_{k,a} \dot{R}_k C_{k,b}^\top \right. \\ & \left. + C_{k,a} R_k \dot{C}_{k,b}^\top \right], \end{aligned} \quad (74)$$

with $\dot{R}_D = 0$. Therefore

$$\hat{z}_t = \dot{R}_0 + \dot{\tau}_t = \dot{R}_0. \quad (75)$$

For the marginal numerator, differentiate the square core:

$$\dot{H}_k(z_k) = \dot{G}_k(z_k) \otimes G_k(z_k) + G_k(z_k) \otimes \dot{G}_k(z_k). \quad (76)$$

Integrated environments are differentiated by the concrete chain rule (61). In array form, the stored matrices have shapes

$$Q_k, \dot{Q}_k, E_k, \dot{E}_k \in \mathbb{R}^{r_{k-1}^2 \times r_k^2}, \quad P_k \in \mathbb{R}^{1 \times r_k^2}, \quad S_k \in \mathbb{R}^{r_k^2 \times 1}. \quad (77)$$

The defensive kept term is fixed, so its derivative is zero in the primary P15 branch.

Proposition 6 (Normalized approximate filtering). *If ϕ_t is measurable, $\tau_t > 0$, and $R_0 < \infty$, then*

$$\hat{p}_t(z_K) = a_t(z_K)/\hat{z}_t \quad (78)$$

is a nonnegative normalized density on the kept coordinates.

Proof. The density $\hat{q}_t = \phi_t^2 + \tau_t 2^{-D}$ is nonnegative and has finite positive integral $\hat{z}_t = R_0 + \tau_t$. Hence $\hat{q}_t/\hat{z}_t$ is a normalized joint density on $[-1, 1]^D$. The marginal over z_K is nonnegative and integrates to one by Tonelli's theorem. $\square$

Proposition 7 (Same-scalar fixed-branch gradient). *For the saved branch B , the gradient algorithm differentiates exactly*

$$\widehat{\ell}_T^{\text{TT}}(\alpha; B) = \sum_{t=1}^T \{\log(R_{t,0}(\alpha; B_t) + \tau_t) - c_t\}. \quad (79)$$

For one parameter coordinate,

$$\partial_i \widehat{\ell}_T^{\text{TT}}(\alpha; B) = \sum_{t=1}^T \frac{\partial_i R_{t,0}(\alpha; B_t)}{R_{t,0}(\alpha; B_t) + \tau_t}. \quad (80)$$

Proof. For the primary branch, $\dot{\tau}_t = \dot{c}_t = 0$. Differentiating the t -th summand gives

$$\partial_i \{\log(R_{t,0} + \tau_t) - c_t\} = \frac{\dot{R}_{t,0}}{R_{t,0} + \tau_t}. \quad (81)$$

The cores that determine $R_{t,0}$ are obtained by a fixed finite sequence of differentiable ridge solves. Proposition 3 gives the derivative of each solve, and (74) gives the derivative of the mass contraction. The previous-filter dependence is included through (66). Therefore the derivative is the derivative of the same scalar computed by the forward pass. $\square$

15 Complete Gradient Algorithm In Pseudocode

- G1.** Run the forward filter and save every branch object, ALS system, core, environment, R_0 , $\widehat{z}_t$, and marginal numerator object.
- G2.** For a parameter coordinate α_i , evaluate $\partial_i \log f_\alpha$ and $\partial_i \log g_\alpha$ at fitting points.
- G3.** At $t = 1$, use the prior derivative. At $t > 1$, evaluate $\partial_i \log \widehat{p}_{t-1}$ by (66).
- G4.** Form $\dot{y}_j$ by (64).
- G5.** Replay the fixed ALS sweeps. At every core update, form $\dot{A}$, $\dot{N}$, $\dot{d}$, and solve (69).
- G6.** Compute $\dot{R}_0$ by (74).
- G7.** Add $\dot{R}_0/(R_0 + \tau_t)$ to the score.
- G8.** Build derivative environments for the next **FilterObject**: save $\mathcal{H}_{k,ab}$, $\dot{\mathcal{H}}_{k,ab}$, Q_k , $\dot{Q}_k$, $\widehat{z}_t$, $\dot{\widehat{z}}_t$, and $\tau_t 2^{-|K|}$. Its evaluation routine is exactly **E1–E5**.

This recipe is intentionally replay based. It is longer than simply applying automatic differentiation, but every derivative is tied to a stored object in the forward pass.

16 Numerical Stabilization And Failure Diagnostics

The implementation must report:

$$\frac{\|Ag - y\|_2}{\|y\|_2}, \quad \max_k r_k, \quad \max_{\text{ALS updates}} \kappa(N_k), \quad \frac{\tau_t}{R_0 + \tau_t}. \quad (82)$$

It must also report whether any fitted or marginal numerator evaluation is nonfinite. A large residual means the square-root density was not fitted well. A large condition number means the fixed linear algebra is unstable. An overly large defensive-mass ratio means the uniform floor is dominating the filter. These diagnostics can veto a run, but they do not prove posterior accuracy when they look good.

17 Finite-Difference Parity Test

For the saved branch B_0 , test the scalar

$$\widehat{\ell}_T^{\text{TT}}(\alpha; B_0) = \sum_t \{\log(R_{t,0}(\alpha; B_0) + \tau_t) - c_t\}. \quad (83)$$

For each tested coordinate i , compute

$$D_i(h) = \frac{\widehat{\ell}_T^{\text{TT}}(\alpha_0 + he_i; B_0) - \widehat{\ell}_T^{\text{TT}}(\alpha_0 - he_i; B_0)}{2h}, \quad (84)$$

using

$$h \in \{10^{-3}, 10^{-4}, 10^{-5}, 10^{-6}\} \max(1, |\alpha_{0,i}|). \quad (85)$$

Report

$$e_i(h) = \frac{|D_i(h) - \partial_i \widehat{\ell}_T^{\text{TT}}(\alpha_0; B_0)|}{1 + |D_i(h)| + |\partial_i \widehat{\ell}_T^{\text{TT}}(\alpha_0; B_0)|}. \quad (86)$$

The branch objects, fitting points, ranks, shift, defensive mass, and sweep count must remain unchanged at $\alpha_0 \pm he_i$. If the smallest $e_i(h)$ is larger than 10^{-4} in the minimal example, the reference implementation fails the same-scalar test.

18 Minimal Runnable Example

The P15 reference script implements a two-step scalar nonlinear model:

$$x_0 \sim N(0, \sigma_0^2), \quad x_1 | x_0 \sim N(\alpha x_0, \sigma_\eta^2), \quad (87)$$

$$y_1 | x_1 \sim N(x_1^2, \sigma_\epsilon^2). \quad (88)$$

The second step uses the saved approximate filter from the first step:

$$\widehat{p}_1(x_1; \alpha) = a_1(x_1; \alpha) / \widehat{z}_1(\alpha), \quad x_2 | x_1 \sim N(\alpha x_1, \sigma_\eta^2), \quad y_2 | x_2 \sim N(x_2^2, \sigma_\epsilon^2). \quad (89)$$

Thus the example exercises the recursive carry-forward object, not only the one-step normalizer.

At each step the augmented variable is $r = (x_t, x_{t-1})$, so $D = 2$. The script uses $[-3, 3]^2$, degree 5, $N_{\text{fit}} = 96$, fixed rank one, ridge 10^{-8} , five bidirectional sweeps, and $\epsilon_\tau = 10^{-10}$. The parameter is α .

The derivative of the transformed log density is

$$\partial_\alpha \log \widetilde{q}(z; \alpha) = \frac{(x_1 - \alpha x_0)x_0}{\sigma_\eta^2}, \quad (90)$$

because the prior, likelihood, and fixed Jacobian do not depend on α . The script then applies (64), (69), and (80).

At the second step, the derivative of the transformed log density includes the saved-filter term:

$$\partial_\alpha \log \widetilde{q}_2 = \partial_\alpha \log \widehat{p}_1(x_1; \alpha) + \frac{(x_2 - \alpha x_1)x_1}{\sigma_\eta^2}. \quad (91)$$

The script obtains $\partial_\alpha \log \widehat{p}_1$ from (62). This is the concrete recursive path that the one-step P14 note did not test.

The run produced:

$$\begin{aligned}
c_1 &= -0.445460193510, \\
c_2 &= -1.496961434280, \\
\widehat{\ell} &= -0.56944222488, \\
\partial_\alpha \widehat{\ell} &= -1.004731548060, \\
\text{step 1 fit residual} &= 3.905823 \times 10^{-1}, \\
\text{step 2 fit residual} &= 3.544318 \times 10^{-1}, \\
\max \kappa(N) \text{ at step 1} &= 2.977710, \\
\max \kappa(N) \text{ at step 2} &= 2.914195.
\end{aligned} \tag{92}$$

The centered-difference relative errors were

$$\begin{array}{ccc}
h & D(h) & e(h) \\
10^{-3} & -1.004731310930 & 7.879523 \times 10^{-8} \\
10^{-4} & -1.004731545700 & 7.856375 \times 10^{-10} \\
10^{-5} & -1.004731548040 & 7.604872 \times 10^{-12} \\
10^{-6} & -1.004731548630 & 1.879178 \times 10^{-10}.
\end{array} \tag{93}$$

The parity check passes. The large-ish fit residual is not hidden: this toy example is a same-scalar gradient and implementability check, not a claim that rank one is an accurate filter for the nonlinear density.

19 Relation To Zhao–Cui And What Remains To Be Tested

Zhao–Cui use sequential tensor-train approximations of the joint density over (x_t, θ, x_{t-1}) , squared density representations of the form $\phi^2 + \tau\lambda$, normalizers, marginalization, and conditional transport maps. P15 keeps that filtering architecture but freezes the numerical branch and replaces the adaptive approximation machinery by one explicit fixed-rank ridge ALS path so that a same-scalar derivative can be written and tested.

What this note proves is limited but useful:

$$\text{fixed branch} + \text{squared density} + \text{finite positive mass} \implies \text{normalized approximate filter}, \tag{94}$$

and

$$\text{fixed branch} + \text{differentiable stored solves} \implies \nabla_\alpha \widehat{\ell}_T^{\text{TT}}(\alpha; B) \text{ for the same scalar.} \tag{95}$$

What remains empirical is accuracy and efficiency on a target high-dimensional model: fit residuals, rank growth, conditioning, defensive-mass ratio, finite-difference parity, and comparison against fixed sparse-grid Gaussian projection filtering.

A Source And Code Anchors

The main text is self-contained. These anchors explain provenance:

- Zhao–Cui state-space model equations (1)–(3): source for the SSM notation.
- Zhao–Cui equations (9)–(12) and Algorithm 1: source for the sequential joint density over (x_t, θ, x_{t-1}) and the integration architecture.
- Zhao–Cui equation (13), Lemma 1, Algorithm 2, and Proposition 2: source for squared-TT density and marginal construction.

- Zhao–Cui Section 4.1: source for the evidence/normalizer role.
- Cui–Dolgov Sections 2–3: source context for squared inverse Rosenblatt and functional tensor-train transport.
- Inspected code paths: `models/full_sol.m` updates `log(sirt.z)–const`, and `@TTSIRT/marginalise.m` computes `obj.z = obj.fun.z + obj.tau`.

The exact Legendre basis, Halton design, fixed-rank ridge ALS protocol, frozen shift rule, and reference Python example are P15 design choices. They are not claimed to be Zhao–Cui’s adaptive MATLAB implementation.

B Symbol Table And Array Shapes

Symbol	Meaning
D	augmented dimension of $r_t = (x_t, \vartheta, x_{t-1})$.
z	reference coordinate in $[-1, 1]^D$.
Ψ_t	affine map from reference coordinate to physical coordinate.
$b_{k,a}$	normalized Legendre basis function for coordinate k .
$C_{k,a}$	coefficient slice, shape $r_{k-1} \times r_k$.
$G_k(z_k)$	TT factor, shape $r_{k-1} \times r_k$.
ϕ_t	square-root TT function.
H_k	square core $G_k \otimes G_k$.
R_k	suffix mass-contraction environment.
$A^{(k)}$	design matrix for the ALS update of core k .
N_k	ridge normal matrix $A^\top W A + \rho I$.
a_t	marginal numerator for the next filter.
$\hat{z}_t$	approximate one-step normalizer.
B_t	all frozen branch objects at time t .